CHAROTAR UNIVERSITY OF SCIENCE & TECHNOLOGY

Faculty of Technology and Engineering

Devang Patel Institute of Advance Technology and Research

Department of Information Technology

BIG DATA ANALYTICS (CSUE301)

PRACTICAL 3

Implement Word Count / Frequency Program using MapReduce

Student ID: 24DIT021

Name:-Hit Goyani

1. OBJECTIVE

The objective of this practical is to implement a Word Count program using Hadoop MapReduce. The program reads text files, counts how many times each word occurs and stores the final result in HDFS. This practical also helps in understanding the basic working of Mapper, Reducer and HDFS.

2. AIM

To implement Word Count / Word Frequency program using Hadoop HDFS and MapReduce.

3. PROBLEM DEFINITION

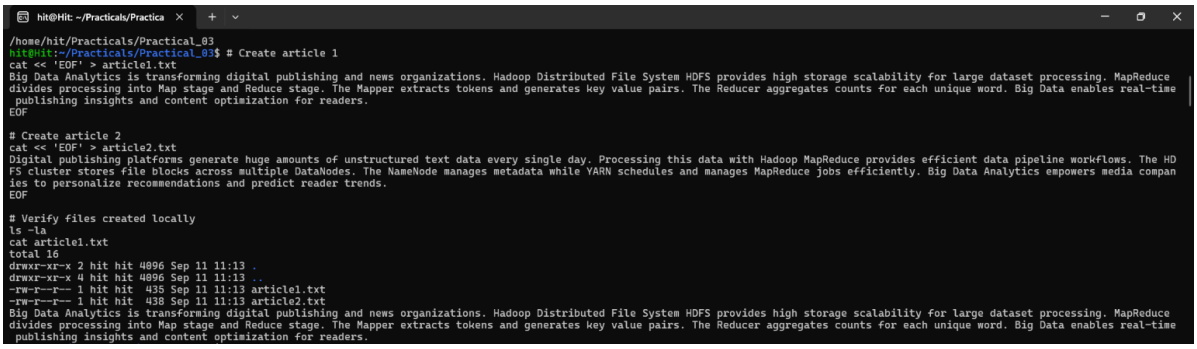
A digital publishing company has a large number of news articles and blog posts. The company wants to know which words are used frequently in the articles. For this, the text files are stored in HDFS and processed using MapReduce. The Mapper reads the words and the Reducer adds their counts to generate the final word frequency.

4. INPUT DATA USED

For this practical, two text files were created:

- article1.txt – contains text related to Big Data, Hadoop, HDFS and digital publishing.
- article2.txt – contains text related to publishing platforms, MapReduce, HDFS, YARN and reader trends.

Both files were used together as input for the MapReduce program.



5. STARTING HADOOP

First, the Hadoop cluster was started. After starting Hadoop, the running Java processes were checked using the jps command. The NameNode, DataNode, SecondaryNameNode, ResourceManager and NodeManager processes were running.

```
hit@Hit: ~/Practicals/Practica
hit@Hit:~$ ls
Downloads  Hadoop  Practicals  Projects  bin  hadoop-3.4.1  hadoopdata
hit@Hit:~$ start-hadoop
======================================
       Starting Hadoop Cluster
======================================

[1/2] Starting HDFS...

[2/2] Starting YARN...

======================================
Running Java Processes
======================================
1216 Jps
914 NodeManager
596 DataNode
517 NameNode
824 ResourceManager
750 SecondaryNameNode

======================================
Hadoop Web Interfaces
======================================
HDFS NameNode       : http://localhost:9870
YARN ResourceManager: http://localhost:8088
NodeManager         : http://localhost:8042
======================================

Opening Hadoop dashboards...

Hadoop started successfully!
hit@Hit:~$ ls
Downloads  Hadoop  Practicals  Projects  bin  hadoop-3.4.1  hadoopdata
hit@Hit:~$ # 1. Check running Hadoop Java processes
jps

# 2. Create and enter Practical 3 directory
mkdir -p ~/Practicals/Practical_03
cd ~/Practicals/Practical_03
pwd
914 NodeManager
596 DataNode
517 NameNode
824 ResourceManager
2811 Jps
750 SecondaryNameNode
/home/hit/Practicals/Practical_03
```

6. UPLOADING INPUT FILES TO HDFS

An input directory was created in HDFS and both article files were uploaded.

```
hdfs dfs -mkdir -p /user/hit/input_p3

hdfs dfs -put article1.txt article2.txt /user/hit/input_p3/

hdfs dfs -ls /user/hit/input_p3/
```

The files article1.txt and article2.txt were successfully visible inside the HDFS input directory.

```
hit@Hit:~/Practicals/Practical_03$ # Create input directory in HDFS
hdfs dfs -mkdir -p /user/hit/input_p3

# Upload local text files to HDFS
hdfs dfs -put article1.txt article2.txt /user/hit/input_p3/

# Verify uploaded files in HDFS
hdfs dfs -ls /user/hit/input_p3/
Found 2 items
-rw-r--r--   1 hit supergroup        435 2026-09-11 11:13 /user/hit/input_p3/article1.txt
-rw-r--r--   1 hit supergroup        438 2026-09-11 11:13 /user/hit/input_p3/article2.txt
hit@Hit:~/Practicals/Practical_03$
hit@Hit:~/Practicals/Practical_03$ cat << 'EOF' > WordCount.java
import java.io.IOException;
import java.util.StringTokenizer;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WordCount {
```

7. MAPPER AND REDUCER PROGRAM

The program was written in Java. The Mapper reads one line at a time, converts the text to lowercase and separates the words. Common stop words were ignored. For every remaining word, the Mapper generates the pair (word, 1).

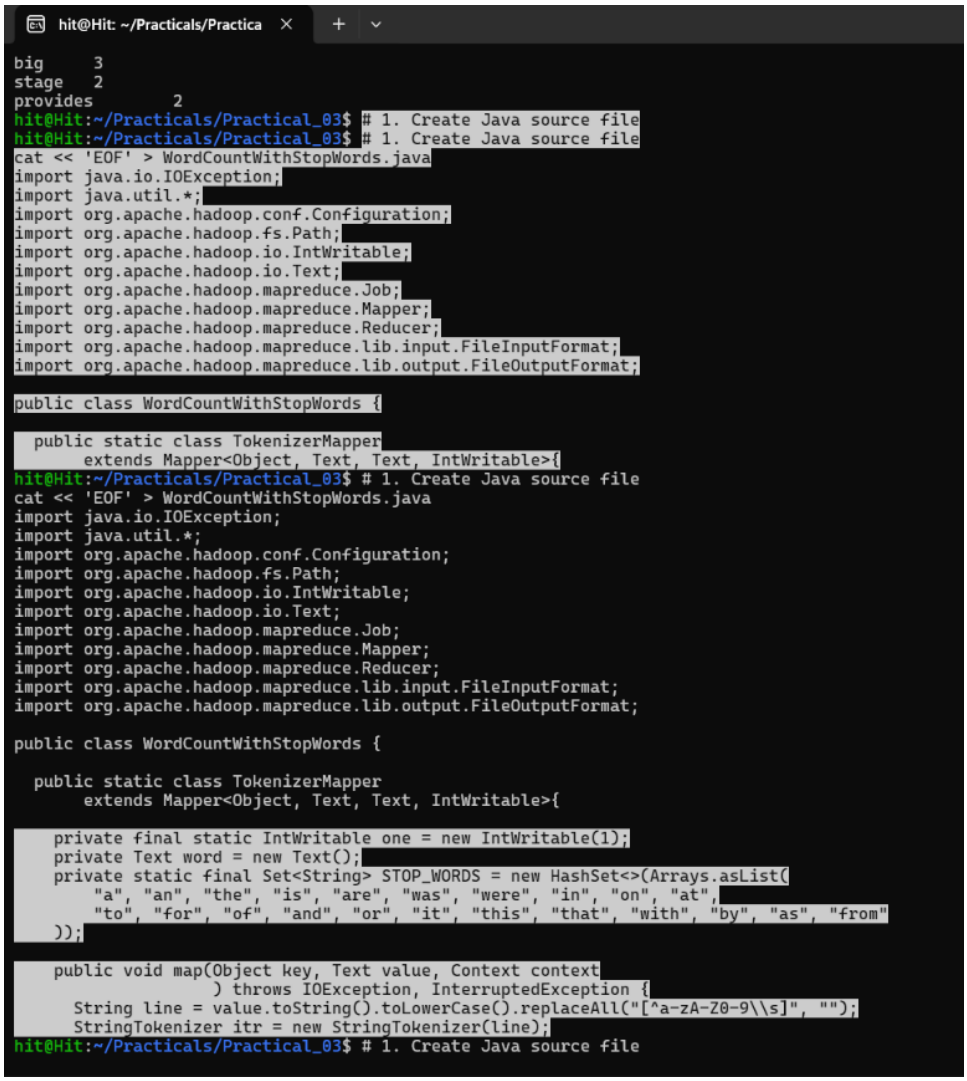
The Reducer receives the same word together with all its values and adds the values. For example, if the Mapper produces (data,1), (data,1) and (data,1), the Reducer produces (data,3).

Mapper Logic

- Read input text.
- Convert text to lowercase.
- Remove punctuation.
- Split the text into words.
- Ignore common stop words.
- Write each valid word with value 1.

Reducer Logic

- Receive the same word and its values.
- Add all the values.
- Write the final word count.


hit@Hit: ~/Practicals/Practica
big 3
stage 2
provides 2
hit@Hit:~/Practicals/Practical_03$ # 1. Create Java source file
hit@Hit:~/Practicals/Practical_03$ # 1. Create Java source file
cat << 'EOF' > WordCountWithStopWords.java
import java.io.IOException;
import java.util.*;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
public class WordCountWithStopWords {
public static class TokenizerMapper
extends Mapper<Object, Text, Text, IntWritable>{
hit@Hit:~/Practicals/Practical_03$ # 1. Create Java source file
cat << 'EOF' > WordCountWithStopWords.java
import java.io.IOException;
import java.util.*;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
public class WordCountWithStopWords {
public static class TokenizerMapper
extends Mapper<Object, Text, Text, IntWritable>{
private final static IntWritable one = new IntWritable(1);
private Text word = new Text();
private static final Set<String> STOP_WORDS = new HashSet<>(Arrays.asList(
"a", "an", "the", "is", "are", "was", "were", "in", "on", "at",
"to", "for", "of", "and", "or", "it", "this", "that", "with", "by", "as", "from"
));
public void map(Object key, Text value, Context context
) throws IOException, InterruptedException {
String line = value.toString().toLowerCase().replaceAll("[^a-zA-Z0-9\\s]", "");
StringTokenizer itr = new StringTokenizer(line);
hit@Hit:~/Practicals/Practical_03$ # 1. Create Java source file

```
hit@Hit:~/Practicals/Practical_03$ cat << 'EOF' > WordCount.java
import java.io.IOException;
import java.util.StringTokenizer;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WordCount {

  // Task 2: Mapper Program
  public static class TokenizerMapper
       extends Mapper<Object, Text, Text, IntWritable>{

    private final static IntWritable one = new IntWritable(1);
    private Text word = new Text();

    public void map(Object key, Text value, Context context
                    ) throws IOException, InterruptedException {
      String line = value.toString().toLowerCase().replaceAll("[^a-zA-Z0-9\\s]", "");
      StringTokenizer itr = new StringTokenizer(line);
      while (itr.hasMoreTokens()) {
        word.set(itr.nextToken());
        context.write(word, one);
      }
    }
  }

  // Task 3: Reducer Program
  public static class IntSumReducer
       extends Reducer<Text,IntWritable,Text,IntWritable> {
    private IntWritable result = new IntWritable();

    public void reduce(Text key, Iterable<IntWritable> values,
                       Context context
                       ) throws IOException, InterruptedException {
      int sum = 0;
      for (IntWritable val : values) {
        sum += val.get();
      }
      result.set(sum);
      context.write(key, result);
    }
  }

  // Driver Code
  public static void main(String[] args) throws Exception {
    Configuration conf = new Configuration();
```

8. COMPILATION AND JAR CREATION

After writing the Java program, it was compiled and a JAR file was created.

```
hadoop com.sun.tools.javac.Main WordCount.java

jar cf wc.jar WordCount*.class

ls -la wc.jar
```

```
hit@Hit:~/Practicals/Practical_03$ # Compile Java source code
hadoop com.sun.tools.javac.Main WordCount.java

# Create Jar file
jar cf wc.jar WordCount*.class

# Check created jar file
ls -la wc.jar
-rw-r--r-- 1 hit hit 3306 Sep 11 11:14 wc.jar
hit@Hit:~/Practicals/Practical_03$ # Remove output folder in HDFS if it already exists
```

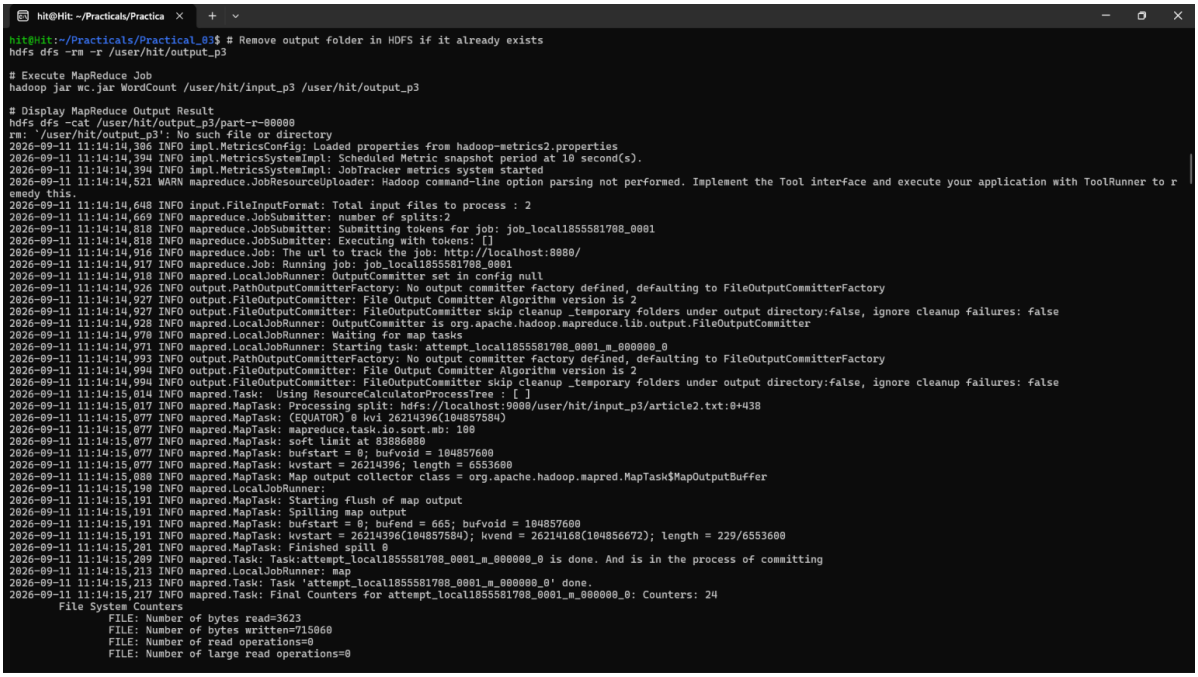
9. EXECUTION OF MAPREDUCE PROGRAM

Before running the program, the old output folder was removed if it already existed. Then the MapReduce job was executed using the input directory and a new output directory.

```
hdfs dfs -rm -r /user/hit/output_p3
```

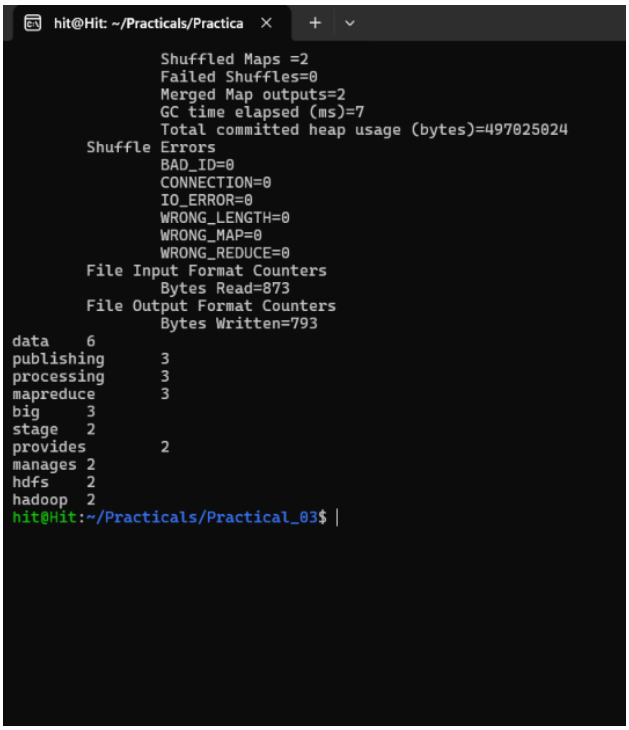
```
hadoop jar wc.jar WordCount /user/hit/input_p3 /user/hit/output_p3
```

```
hdfs dfs -cat /user/hit/output_p3/part-r-00000
```



10. OUTPUT

The MapReduce job completed successfully and the output was stored in HDFS. The output contains each word and its total frequency.



11. TOP 10 FREQUENT WORDS

From the output shown in the execution screenshot, the frequently occurring meaningful words were:

- data – 6
- big – 3
- publishing – 3
- processing – 3
- mapreduce – 3
- hadoop – 2
- hdfs – 2
- manages – 2
- provides – 2
- stage – 2

The word data occurred the highest number of times in the final output.

12. PERFORMANCE OBSERVATION

The MapReduce job was executed successfully on the given two input files. The screenshots show the successful job execution and Hadoop counters. A separate benchmark with small, medium and large datasets was not performed in the available execution record, so exact timing values for different dataset sizes are not included.

In general, for very small files, Hadoop can take extra time because the framework has to start and schedule the job. When the amount of data becomes large, distributed processing becomes more useful.

13. KEY QUESTIONS / ANALYSIS

Q1. What is the role of Mapper and Reducer in the Word Count application?

The Mapper reads the text and generates (word, 1) for each valid word. The Reducer receives the same words together and adds their values to get the final count.

Q2. How are intermediate key-value pairs generated and processed?

For every word found by the Mapper, a pair such as (data, 1) is generated. Hadoop groups the same words during Shuffle and Sort. The Reducer then adds the values of each group.

Q3. How does HDFS store and distribute the input dataset?

HDFS stores files in blocks. In a Hadoop cluster, blocks can be stored on different DataNodes. The NameNode keeps information about the files and blocks. In this practical, Hadoop was running on a single local machine.

Q4. What advantages does MapReduce offer for processing large datasets?

MapReduce can process data in parallel and can work with large datasets. It can also be scaled by adding more nodes. It is useful when the data is too large to process easily on one system.

Q5. How does execution time vary with dataset size?

For small datasets, Hadoop startup and job scheduling can take a noticeable amount of time. As the dataset size increases, the benefit of distributed processing becomes more useful. Exact time comparison was not available from the current practical execution screenshots.

14. TOOLS / TECHNOLOGY USED

- Ubuntu Linux / Terminal
- Hadoop 3.4.1
- HDFS
- YARN
- Java
- Hadoop MapReduce

15. CONCLUSION

In this practical, the Hadoop environment was started successfully and two text files were uploaded to HDFS. A Java MapReduce program was used to count the frequency of words. The Mapper generated the intermediate word and count pairs, while the Reducer added the counts of the same words. Stop words were ignored for better analysis. The final output was generated successfully in HDFS, and the top frequent words were identified.

16. FINAL RESULT

The Word Count / Frequency program using Hadoop MapReduce was implemented and executed successfully. The input files were processed from HDFS and the final word frequency output was generated successfully.